# Meet-up talk outline — "Single Responsibility Principle in AI: one door to the LLM"

*Generated from `deck.html` (the `SLIDES` block) by `node presentation/scripts/deck-outline.js presentation/meetup/deck.html presentation/meetup/outline.md`. Edit notes in the deck, then regenerate.*

Planned length: **47 min** · 24 slides · English.

Deck controls: → / Space / PageDown = next (reveals builds first) · ← = back · P = pause · R = restart animation · – / + = animation speed (0.25×–3×, remembered) · 0 = 1× · F = fullscreen · S = presenter notes window.

## Opening — 2.5 min

**1. About me** *(0.5 min, starts at 0:00)*

- Short intro: who I am, what I do at Spin.AI, why AI-in-a-fleet is a real problem for us — keep it under 30 seconds.

**2. Single Responsibility Principle in AI** *(0.5 min, starts at 0:30)*

- Today: an architecture story about AI inside a fleet of microservices — and a 20-year-old principle that turns out to be very current.
- This is an architecture talk, not a framework talk. No annotations, no versions, no class names on the slides. You can relax your Java muscles.
- Typed Goose is the study project behind the talk — a small system we built end-to-end, mostly together with an AI pair programmer. More on that at the end.

> 😄 If you came for the framework-migration war stories — that's the after-party.

**3. One reason to change** *(1.5 min, starts at 1:00)*

- Optional quick poll (verbal, not on the slide): "Who has an LLM API key in the config of more than one service?" Count hands. "Keep your hand up if you know exactly how many." Hands usually drop — "…that's what I thought." That is the talk in one gesture.
- SRP: one reason to change. Billing changing because tax rules changed — fine, that is its job. Billing changing because an LLM vendor renamed a model — that is a second reason to change.
- That second reason, multiplied by every service that talks to an LLM, is the smell we are chasing today.

> 😄 Uncle Bob gave us SRP in the early 2000s. Twenty years later we gave every service a second reason to change: an API key.

## Part 1 · The problem — 11.5 min

**4. Meet the fleet** *(2.5 min, starts at 2:30)* 🎞 **looping animation**

- A typical company: 20–30 services, owned by different teams, different release cycles. Each one has one job. Billing bills. Search searches. Support… supports.
- This is the clean picture — and the hidden dependencies everyone knows about but nobody draws.
- And now product walks in with the question of the decade: "Can we add AI?"

> 😄 Our architecture diagram — as presented to management. …And as it actually is.

**5. The first one** *(3 min, starts at 5:00)* **looping animation**

- Support wants to summarize long tickets. One team, one service, one line to an LLM provider.
- They put the key into an env var, hard-code the model name, write a little retry loop. The bill goes to… somebody's credit card.
- And honestly — for ONE service this is the right call. Do not build a platform for one caller.
- But notice the red robot: Support now has two jobs. Supporting customers, and being an LLM client.

> 😄 It took a sprint. The demo went great. The PM cried. Happy tears — for now.

**6. The second, the third, the fifth…** *(3 min, starts at 8:00)* **looping animation**

- Success is contagious. Search wants semantic search, Reports wants narrative summaries, Marketing wants copy, Docs wants Q&A.
- Everybody copy-pastes Support's code — or doesn't, and writes their own. Marketing read a blog post, so they use a different provider. Of course.
- Five keys in five vaults, three SDK versions, two-three providers, and nobody can say what the monthly bill is.
- Story beat (animation): Reports runs a nightly job that hammers the shared key. The provider says 429. And who gets the error? Support's customer-facing summary. At 2 a.m.

> 😄 Microservices: because one big ball of mud wasn't distributed enough.

**7. Dozens** *(3 min, starts at 11:00)* **looping animation**

- Fast-forward a year. Twenty services talk to LLMs — some to two providers. Count the keys: 23.
- Key rotation = 23 pull requests across 15 teams. A model gets deprecated = fleet-wide fire drill.
- Finance asks: how much do we spend on AI, per team? Nobody knows. Compliance asks: do we send customer PII to an LLM? 23 code bases to audit.
- Every team picked its own timeouts and retries. Some retry forever. On a paid API.
- This is an SRP violation at fleet scale: the "LLM client" responsibility is copy-pasted into every service, so every vendor change becomes a change in every service.
- Five services survived untouched: Auth, Shipping, Invoices, HR, Audit — the only teams that read the security policy.

> 😄 When a provider deprecates a model, you find out which teams read release notes. Spoiler: none.

## Part 2 · One door — 6.5 min

**8. One door: the AI Gateway** *(2.5 min, starts at 14:00)* **looping animation**

- The fix is the oldest trick in the book: take the responsibility that is duplicated everywhere and give it ONE owner.
- An AI Gateway is a service whose single job is talking to LLMs. Everything else in the fleet goes back to its own job.
- Look at the chips: keys, provider abstraction, model tiers, timeouts, rate limits, cost, observability, sync and batch. Many features — but one responsibility: being a good LLM client.
- Callers speak business ("summarize this, fast tier"), not vendor ("model X, temperature 0.2, header Y").

> 😄 We didn't remove the complexity. We gave it a home address.

**9. Pros & cons — honestly** *(2 min, starts at 16:30)*

- Single point of failure: yes. So you run more than one instance and treat it like any other critical service.
- Extra hop: a few milliseconds in front of a call that takes seconds. Nobody will notice.
- The god-gateway risk is the SRP point again: the day someone adds "if tenant is ACME, use the special prompt" into the gateway, the violation did not disappear — it just moved house. Business prompts belong to the business service.

> 😄 We replaced 23 problems with one. It's a very big one — but at least it's ours.

**10. A solution — not the solution** *(2 min, starts at 18:30)*

- Be explicit: this is not the one true way. It is the one we built to learn what it takes.
- Library: zero hops, but every change is N upgrades, and the library has to exist in every language you use.
- Off-the-shelf proxies are a perfectly legit "buy before build".
- Transition: in our study project we built one, with the two kinds of traffic you'll always meet — a human waiting (chat) and a machine waiting (file processing).

> 😄 Architecture answer #1: "it depends". Architecture answer #2: "it still depends — but now with a diagram".

## Part 3 · Sync: chat — 5.5 min

**11. Sync flow ① — the question goes in** *(2 min, starts at 20:30)* 🎞️ **looping animation**

- Only the business pieces: UI, Chat Service, AI Gateway, LLM provider. Each box has exactly one job — it is written under its name.
- The UI renders. The Chat Service owns the conversation: it adds history and makes sure there is only one reply in flight per chat.
- The AI Gateway owns the LLM call: it picks the provider and the model tier. The Chat Service never knows the vendor's model name.
- The speed control (– / +) is handy here if the audience wants to see the hops slowly.

> 😄 Note the question. We only test with real-world use cases.

**12. Sync flow ② — the answer streams back** *(2 min, starts at 22:30)* 🎞️ **looping animation**

- The LLM produces the answer token by token. We do not wait for the full answer at any hop — each chunk is forwarded the moment it arrives.
- Humans judge speed by time-to-first-word, not total time. Streaming turns 8 seconds of silence into 8 seconds of reading.
- The Chat Service relays the chunks AND accumulates the full reply, so it can store the conversation history at the end.
- War story: we once had perfect streaming through all four hops — and the UI didn't scroll, so the answer looked like it arrived all at once. We measured every hop before realizing the bug was the scroll bar.

> 😄 Streaming: a loading spinner that actually tells you something.

**13. Sync flow — the whole round-trip** *(1.5 min, starts at 24:30)* 🎞️ **looping animation**

- Both halves together. Every hop is a stream; nobody waits for the full answer.

- SRP check: swap the LLM provider — only the AI Gateway changes. Change the chat history rules — only the Chat Service changes. Redesign the UI — only the UI changes.

> 😄 Four boxes, four jobs, zero API keys in the Chat Service. Uncle Bob would be proud. Or at least less disappointed.

## Part 4 · Async: files — 7 min

**14. Async flow ① — submit & wait** *(2.5 min, starts at 26:00)* **looping animation**

- Second kind of traffic: nobody is staring at a spinner. Summarize a pile of uploaded files.
- The Calculation Service owns jobs and files. It saves the job as PENDING, returns a job id immediately, and drops one message per file on the message broker.
- The UI polls for status. Polling is boring — and that is a feature: stateless, cache-friendly, good enough for jobs that take minutes.
- Notice: the Calculation Service has no idea an LLM exists. It just put work on a queue.

> 😄 The UI polls like a kid in the back seat: are we there yet? Are we there yet?

**15. Async flow ② — batch processing** *(3 min, starts at 28:30)* **looping animation**

- The AI Gateway pulls messages from the broker and collects them into a batch — "5 messages or a few minutes, whichever comes first".
- It sends ONE batch to the provider's Batch API. Batch is roughly half the price of live calls; the trade-off is time — minutes, sometimes hours.
- Then it waits and checks the batch status periodically. It remembers open batches, so a restart doesn't lose them.
- Results go back onto the broker, the Calculation Service stores them, and the next UI poll finally says DONE.

> 😄 Batch API: 50% cheaper, delivered sometime between now and your next stand-up.

**16. Async flow — the whole round-trip** *(1.5 min, starts at 31:30)* **looping animation**

- Whole thing, with a time-lapse clock: in real life the gap in the middle is minutes. The gap IS the point — that's where the 50% saving lives.
- SRP again: UI renders and asks; Calculation Service owns jobs; DB remembers; broker delivers; AI Gateway talks to the LLM. Switching to another provider's batch API touches one service.

> 😄 Async is just sync with trust issues resolved.

## Part 5 · The whole system — 9 min

**17. The whole system — platform pieces** *(2.5 min, starts at 33:00)* **looping animation**

- Now the real picture. Nginx is the front door: it serves the UI and forwards API calls. The API Gateway routes requests to services by name.
- Eureka is the phone book: services register when they start; the API Gateway and the Chat Service ask it "where is chat-service?" / "where is the AI gateway?".
- Postgres plays "the database", Kafka plays "the message broker". Calc, Kafka and Postgres stay idle here — this animation is the chat flow.
- Yes: API Gateway and AI Gateway. One letter apart, completely different jobs. Routing vs. talking to LLMs.

> 😄 Eureka: the only service named after how you feel when it finally works.

**18. Watching it — logs, traces, health** *(2 min, starts at 35:30)* 🎞️ **looping animation**

- Business traffic is dimmed on purpose. This is the "who watches the watchers" layer, and each piece has one job too.
- Logs: every service writes structured JSON logs → Logstash → Elasticsearch; Kibana is where humans search them.
- Traces: every hop reports a span to Jaeger, which stitches them into one trace. The first time you open it you see the LLM taking ~95% of the time.
- Spring Boot Admin asks Eureka who exists, then checks each service's health — one dashboard of green (hopefully) lights.

> 😄 Logs tell you what happened. Traces tell you where. Nobody tells you why.

**19. Everything at once** *(1.5 min, starts at 37:30)* 🎞️ **looping animation**

- Chat, files and observability all running together. About fifteen containers, and it all comes up on a laptop with one command.
- Optional: switch to the live demo here — send a chat message, upload two files, open the trace in Jaeger.

> 😄 Also running, not on the diagram: my laptop fan — the loudest service in the architecture.

**20. Live: Typed Goose** *(3 min, starts at 39:00)*

- This is the real app, running on this laptop — the same system as the diagrams.
- Chat: send one message and watch the answer stream in, word by word.
- Files: upload two files, start the summary, and watch the job go from PENDING to DONE (batch can be slow — move on and come back if needed).
- Click on the dark area outside the frame to get the arrow keys back. The ‹ › buttons always work.

## Built with AI — 3 min

**21. How it was built: an AI pair, with a diary** *(1.5 min, starts at 42:00)*

- The project was built with an AI coding assistant as a pair programmer — not a vending machine you throw prompts into.
- The core problem: every session starts with an empty head. So everything important is written down, in the repo, in files the AI reads first.
- Project instructions = the house rules. Steps = the plan, sliced so that one session finishes one step. ADRs = why we decided things. Changelog = what actually happened.

**22. The cross-session loop** *(1.5 min, starts at 43:30)* 🎞️ **looping animation**

- One loop per session: read the next step → brainstorm with the human → plan → implement → verify → update the step file and changelog → stop.
- Verify means the real thing: build green, containers up, AND a real request through the feature. Watch the packet bounce once at "Verify" — that happened a lot.
- Side channel: whenever something demo-worthy happened we saved a curated note; every session end auto-saved the raw transcript. This talk is literally built from those notes.
- SRP for the AI: one session, one step, one responsibility.

> 😄 Stopping is a feature. The hardest prompt to write is "that's enough for today".

## Wrap-up — 2 min

**23. Takeaways** *(1.5 min, starts at 45:00)*

- Four takeaways, one sentence each. Pause after the first one — it is the title of the talk.

**24. Thank you** *(0.5 min, starts at 46:30)*

- Thank the organizers. Take questions. If the room is quiet, offer the live demo.